

Delivery 1 KPIs

In this exercise, you will build a **basic game analytics pipeline** from scratch. The goal is to simulate how commercial games track player activity, send it to a backend server, store it in a database, and analyze the data to calculate **key performance indicators (KPIs)**. You will also practice **statistical analysis in R**, adding **confidence intervals** to your KPIs and predictions.

Part 1 – Data Collection (Unity → Server → MySQL)

- Design the data model that the server will store.
- Implement scripts inside your Unity game to log gameplay data and send it to a server.
- Implement a backend script (e.g., in PHP) to receive this data and insert it into a MySQL database.
- Set up all the needed tables in MySQL

Part 2 – KPI Extraction and Statistical Analysis

Once your data is stored, you should be able to query it to compute standard KPIs in game analytics, including (but not limited to):

Number of users: DAU, MAU

Retention: Stickiness, D1,D3,D7

Monetisation: ARPU, ARPPU

Sessions: n°, length

Combine these KPIs with demographic information of the users, are there trends?

Deliverables

1. **Unity client script(s):** Data collection and transmission.
2. **Server-side code (e.g., PHP):** Receives and inserts events into MySQL.
3. **Database schema:** Designed to be **clean, normalized, and scalable**.
4. **SQL queries:** For extracting the KPIs listed above.
5. **R scripts:** For statistical analysis, confidence intervals, and visualizations.
6. **Presentation (10 min):**
 - Describe your data model and why you structured it that way.
 - Present KPI calculations and visualizations (with confidence levels and predictions)
 - Reflect on the strengths and limitations of your pipeline.

Evaluation criteria

1. Gather & Store (50%)

Focus: Data pipeline design, scalability, and implementation quality.

- **Excellent (8-10)**
 - Data collection schema is well thought out.
 - Database is normalized, scalable, and efficient (indexes, minimal redundancy, logical structure).
 - Data transmission is non-intrusive: lightweight, asynchronous, minimal risk of harming gameplay experience.
 - Code (Unity + server-side) follows SOLID principles, is modular, well-structured, and documented.
 - System can scale to large volumes of players/events without major bottlenecks.
- **Good (6-8)**
 - Data schema is mostly complete but may lack flexibility or scalability for future expansion.
 - Database works but shows minor redundancies or inefficiencies.
 - Data collection is functional but may add some overhead to gameplay.
 - Code structure is clear, but some principles of maintainability or modularity are missing.
- **Fair (4-6)**
 - Data schema captures only the basics, missing key attributes for analysis.
 - Database design is flat or repetitive, with little thought to growth.
 - Data collection could interfere with gameplay performance.
 - Code is working but monolithic or hard to maintain.
- **Poor (0-4)**
 - Data model is inconsistent or incomplete, making KPI calculation unreliable.

- Database is disorganized, unscalable, or missing critical fields.
 - Transmission is unreliable, blocking, or likely to cause gameplay issues.
 - Code is unstructured, buggy, or not submitted.
-

2. Analytics (30%)

Focus: KPI extraction, statistical rigor, correctness of findings.

- **Excellent (8-10)**

- KPIs (DAU, MAU, ARPU, retention, stickiness, etc.) are correctly computed and clearly explained.
- SQL queries are efficient, well-structured, and reusable.
- Analysis in R includes confidence intervals, hypothesis testing, or predictive models, ensuring results are statistically proven.
- Student demonstrates understanding of limitations and uncertainty in the data.

- **Good (6-8)**

- Most KPIs are correctly calculated, with minor issues in query design or interpretation.
- Statistical analysis is present but limited to basic measures (e.g., averages, standard deviation) with minimal treatment of uncertainty.
- Results are correct but not always backed up with strong statistical proof.

- **Fair (4-6)**

- KPI calculations are incomplete, contain noticeable errors, or are not scalable.
- SQL queries are inefficient or overly complex.
- Little to no statistical treatment of results; confidence intervals or tests are missing.
- Findings are more descriptive than analytical.

- **Poor (0-4)**

- KPIs are missing or incorrectly calculated.
 - SQL queries fail to produce meaningful results.
 - No evidence of statistical rigor; conclusions are unsubstantiated.
 - Analysis is superficial or absent.
-

3. Reporting (20%)

Focus: Communication, clarity, and presentation quality.

- **Excellent (8-10)**

- Report is well-structured, concise, and logically organized.
- Results are presented with clear visualizations (e.g., graphs, charts, tables) that enhance understanding.
- Language is precise and free of major errors.
- Presentation is visually appealing, using professional formatting and design.
- Findings are clearly interpreted and contextualized in terms of game business/engagement impact.

- **Good (6-8)**

- Report is organized and understandable but may lack polish.
- Visualizations are present but could be clearer or more impactful.
- Interpretation of results is correct but not deeply insightful.

- **Fair (4-6)**

- Report is fragmented, difficult to follow, or lacks flow.
- Visuals are basic or poorly chosen (e.g., raw tables instead of graphs).
- Explanations are minimal or vague.

- **Poor (0-4)**

- Report is incomplete or incoherent.
- No visualizations, or visuals do not support findings. Or results are presented without explanation or context.